Precision UI Hybrid Automation Framework

Team Name: Elite Automators

Institution: Sagi Rama Krishnam Raju Engineering College

Team Members:

1. **Name:** Jogi Doni Naga Siva
   Branch: Computer Science and Engineering
   Email: jogidoni17@gmail.com

2. **Name:** Kothapalli Vamsi Krishna Varma
   Branch: Computer Science and Engineering
   Email: kvamsikrishnavarma@gmail.com

Project Link: GitHub

TABLE OF CONTENTS

1. Abstract 3
2. Introduction 4
3. Problem Statement 5
4. Objectives 6
5. Technical Stack 7
6. System Architecture 8
7. Project Features 9 - 10
8. System Implementation 11- 23
    8.1 Project Structure setup 11
    8.2 Driver Initialization 11 - 12
    8.3 Page Object Model (POM) 13 -14
    8.4 Test Case Implementation 14 - 15
    8.5 Utility Class Implementation 15-16
    8.6 Waits For Synchronization 16 - 17
    8.7 Capturing Screenshots 17
    8.8 Data Driven Testing 18
    8.9 Logging And Reporting 19 - 23
    8.10 Cross browser Execution 23
9. Results 24
10. Conclusion 25
11. Team Contribution 26
12. Future Enhancements 27

ABSTRACT

This project presents the design and implementation of a Hybrid UI Automation Testing Framework developed for automating key functionalities of the IPL website. The framework is built using Java, Selenium WebDriver, TestNG, and Maven, following the Page Object Model (POM) design pattern to ensure modularity, reusability, and maintainability of the codebase.

The framework focuses on automating critical user scenarios such as footer link validation, team details verification, points table validation, news search functionality, venue verification, auction data validation, and statistics page testing. It incorporates utility classes for handling common operations like element interactions, dynamic waits, configuration management, and JSON-based data-driven testing.

To enhance reporting and debugging, the framework integrates Extent Reports for structured test reporting and Log4j for logging execution details. It also supports cross-browser testing using Chrome and Edge browsers through a flexible driver management system. Screenshots are captured automatically for failed test cases to assist in failure analysis.

Overall, this framework demonstrates a scalable, efficient, and industry-standard approach to UI automation testing, ensuring reliability, better test coverage, and ease of future enhancements.

INTRODUCTION

With the rapid growth of web applications, ensuring their functionality, performance, and reliability has become essential. Manual testing, although effective for small-scale applications, is time-consuming, error-prone, and not scalable for modern dynamic websites. To overcome these challenges, automation testing frameworks are widely used in the software industry to improve efficiency, accuracy, and test coverage.

This project focuses on building a Hybrid UI Automation Testing Framework for the IPL (Indian Premier League) website. The framework is developed using Java, Selenium WebDriver, TestNG, and Maven, following the Page Object Model (POM) design pattern. The primary goal is to automate real-world user scenarios and validate critical functionalities of the website in a structured and maintainable way.

The framework is designed with a layered architecture that separates test logic, page elements, and reusable utilities, making it easy to scale and maintain. It includes features such as dynamic waits, data-driven testing using JSON, cross-browser execution, logging, screenshot capture, and detailed reporting. These features ensure robust test execution and simplify debugging and analysis.

By implementing this framework, the project demonstrates how automation testing can significantly enhance software quality, reduce manual effort, and provide faster feedback during development cycles. It also reflects industry-standard practices used in real-time automation projects.

PROBLEM STATEMENT

Modern web applications like the IPL website contain multiple dynamic features such as live updates, interactive pages, and frequently changing data. Ensuring the correctness and reliability of such applications through manual testing becomes challenging, time-consuming, and prone to human errors. Repeated execution of test cases for every update further increases effort and reduces efficiency.

Additionally, the absence of a structured testing approach can lead to poor test coverage, difficulty in maintaining test cases, and lack of proper reporting and debugging mechanisms. There is also a need to validate data accuracy (such as team details, points table, and venues) and ensure consistent behavior across different browsers.

To address these challenges, there is a requirement to develop a robust and scalable automation testing framework that can efficiently validate key functionalities of the IPL website. The framework should support reusable components, data-driven testing, cross-browser execution, proper logging, and detailed reporting, while reducing manual effort and improving overall testing reliability.

OBJECTIVES

The primary objective of this project is to design and develop a scalable and maintainable UI Automation Testing Framework for validating key functionalities of the IPL website. The framework aims to improve testing efficiency, accuracy, and reusability by following industry-standard practices.

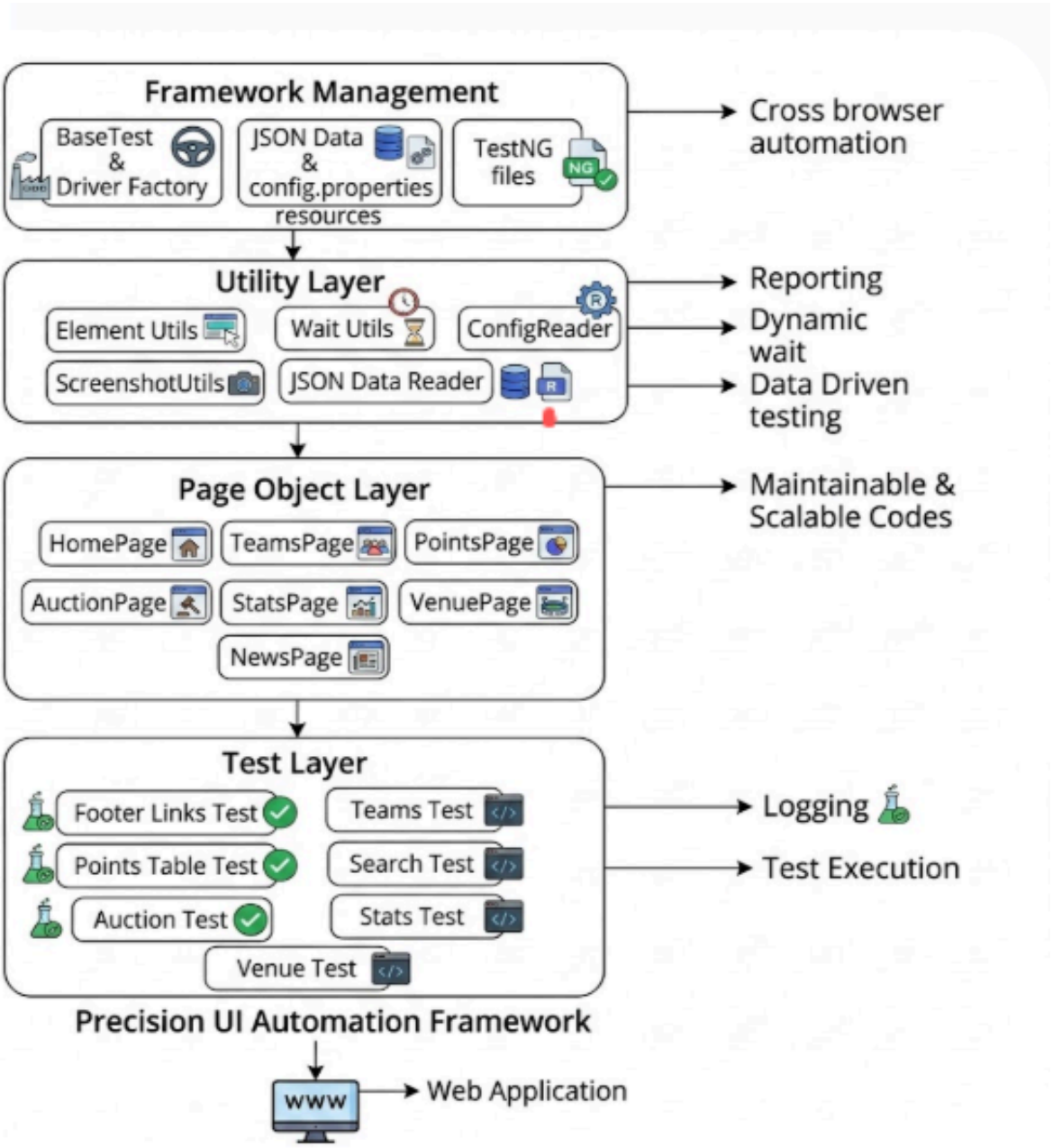
1. **To automate critical user workflows**
   Automate major functionalities such as footer validation, team details verification, points table validation, search functionality, and other core modules of the IPL website.
2. **To implement Page Object Model (POM)**
   Structure the framework using POM to separate test logic from page elements, ensuring better code organization and maintainability.
3. **To achieve reusability through utility classes**
   Develop reusable components for common actions like element interaction, waits, configuration handling, and data management.
4. **To enable data-driven testing**
   Use JSON-based data sources to validate application data dynamically and improve test flexibility.
5. **To support cross-browser testing**
   Ensure the framework can execute tests on multiple browsers such as Chrome and Edge for better compatibility validation.
6. **To implement logging and reporting mechanisms**
   Integrate Log4j for logging and Extent Reports for generating structured and detailed test execution reports.
7. **To reduce manual testing effort and improve efficiency**
   Minimize repetitive manual testing by automating test execution and ensuring faster and more reliable results.

TECHNICAL STACK:

The automation framework is built using modern tools and technologies that are widely used in the software testing industry. The selected stack ensures scalability, maintainability, and efficient test execution.

- Programming Language- Java (JDK 17)
- Automation Tool - Selenium WebDriver (v4.x)
- Testing Framework - TestNG
- Build Tool - Maven
- Design Pattern - Page Object Model (POM)
- Browser Management - WebDriverManager
- Logging Framework - Log4j
- Reporting Tool - Extent Reports
- Data Handling - JSON (Gson Library)
- Version Control - Git

SYSTEM ARCHITECTURE:



PROJECT FEATURES:

The developed UI Automation Framework includes several advanced features that ensure efficiency, scalability, and maintainability of the testing process.

1. PAGE OBJECT MODEL (POM) DESIGN

- Implements POM to separate page locators and actions from test logic.
- Improves code readability, reusability, and maintainability.

2. MODULAR AND LAYERED ARCHITECTURE

- Divides the framework into Base, Page, Utility, and Test layers.
- Ensures clear separation of concerns and easier scalability.

3. CROSS-BROWSER TESTING SUPPORT

- Supports execution on multiple browsers such as Chrome and Edge.
- Uses WebDriverManager for automatic driver setup.

4. DATA-DRIVEN TESTING

- Uses JSON files to store test data such as team details and venues.
- Enables dynamic validation and reduces hardcoding.

5. DYNAMIC WAIT HANDLING

- Uses explicit waits to handle dynamic web elements.
- Improves test stability and reduces flakiness.

6. REUSABLE UTILITY CLASSES

- Provides reusable methods for common actions like click, type, hover, and waits.
- Reduces code duplication and enhances maintainability.

7. LOGGING MECHANISM

- Integrates Log4j to capture detailed execution logs.
- Helps in debugging and tracking test execution flow.

8. ADVANCED REPORTING

- Generates detailed test execution reports using Extent Reports.
- Displays test status, execution time, and logs in a structured format.

9. SCREENSHOT CAPTURE ON FAILURE

- Automatically captures screenshots when test cases fail.
- Assists in quick issue identification and debugging.

10. TESTNG INTEGRATION

- Uses TestNG for test execution, assertions, and test configuration.
- Supports annotations, grouping, and parallel execution capability.

11. CONFIGURABLE FRAMEWORK

- Uses a configuration file (config.properties) to manage environment settings like base URL.
- Makes the framework flexible and easy to modify.

12. SCALABLE AND MAINTAINABLE DESIGN

- New test cases and features can be easily added without affecting existing code.
- Follows industry best practices for long-term usability.

SYSTEM IMPLEMENTATION:

The UI Automation Framework is implemented using a structured and layered approach to ensure scalability, maintainability, and efficient test execution. The implementation follows the Page Object Model (POM) design pattern and integrates various components such as driver management, utility classes, reporting, and data handling.

1. PROJECT STRUCTURE SETUP

The project is organized using the standard Maven directory structure:

- src/main/java → Contains base classes, page objects, and utility classes
- src/test/java → Contains test scripts
- src/test/resources → Contains test data (JSON files)
- testng → Contains TestNG XML files for execution

This structure ensures compatibility with build tools and CI/CD pipelines.

2. DRIVER INITIALIZATION AND MANAGEMENT:

- The DriverFactory class initializes WebDriver instances for different browsers (Chrome and Edge).
- Uses **ThreadLocal** to manage drivers for parallel execution.
- BaseTest class handles setup and teardown using TestNG annotations (@BeforeMethod, @AfterMethod).
- The application URL is fetched dynamically using ConfigReader.

BASETEST:

```
public class BaseTest {
   public WebDriver driver;

   @Parameters("browser")
   @BeforeMethod
   public void setUp(String browser) {

      driver = DriverFactory.initDriver(browser);
```

```
        driver.get(ConfigReader.get("baseURL"));
    }
    @AfterMethod
    public void tearDown(ITestResult result) {

        // If test fails → take screenshot
        if (result.getStatus() == ITestResult.FAILURE) {
            ScreenshotUtils.capture(driver, result.getName());
        }
        DriverFactory.quitDriver();
    }
}
```

DRIVER FACTORY:

```
public class DriverFactory {

    public static ThreadLocal<WebDriver> driver = new ThreadLocal<>();

    public static WebDriver initDriver(String browser) {

        if (browser.equalsIgnoreCase("chrome")) {
            WebDriverManager.chromedriver().setup();
            driver.set(new ChromeDriver());
        }
        else if (browser.equalsIgnoreCase("edge")) {
            WebDriverManager.edgedriver().setup();
            driver.set(new EdgeDriver());
        }

        driver.get().manage().window().maximize();
        return driver.get();
    }

    public static WebDriver getDriver() {
        return driver.get();
    }

    public static void quitDriver() {
        driver.get().quit();
        driver.remove();
    }
}
```

3. PAGE OBJECT MODEL IMPLEMENTATION:

- Each web page is represented as a separate class (e.g., HomePage, TeamsPage, PointsPage).
- Page classes contain:
  - Web element locators (By objects)
  - Methods to perform actions (click, scroll, hover, etc.)
- This approach separates UI logic from test logic, improving code maintainability.

CREATION OF OBJECTS:

```
WebDriver driver;
ElementUtils;
WebDriverWait wait;

public HomePage(WebDriver driver) {
   this.driver = driver;
   this.elementUtils = new ElementUtils(driver);
   this.wait = new WebDriverWait(driver, Duration.ofSeconds(10));
}
```

SEPARATION OF LOCATORS AND ACTIONS:

LOCATORS:

```
private final By footer = By.tagName("footer");

private final By teamSection = By.xpath("//*[@id='footer-main']/div[2]/div[1]/div[1]");
private final By aboutSection = By.xpath("//*[@id='footer-main']/div[2]/div[1]/div[2]");
private final By guidelinesSection = By.xpath("//*[@id='footer-main']/div[2]/div[1]/div[3]");
private final By contactSection = By.xpath("//*[@id='footer-main']/div[2]/div[1]/div[4]");
```

ACTIONS:

```
public void scrollToFooter() {
   WebElement footerElement = driver.findElement(footer);

   JavascriptExecutor js = (JavascriptExecutor) driver;
   js.executeScript("arguments[0].scrollIntoView({block: 'end'});", footerElement);
```

```
}

// General method
private List<String> getLinks(By locator) {
    List<String> linksText = new ArrayList<>();

    List<WebElement> links = driver.findElements(locator);

    for (WebElement link : links) {
        linksText.add(link.getText());
    }

    return linksText;
}

// Methods for each section
public List<String> getTeamLinks() {
    return getLinks(teamSection);
}

public List<String> getAboutLinks() {
    return getLinks(aboutSection);
}

public List<String> getGuidelineLinks() {
    return getLinks(guidelinesSection);
}

public List<String> getContactLinks() {
    return getLinks(contactSection);
}
```

4. TEST CASE IMPLEMENTATION

- Test cases are written using TestNG and placed in the tests package.
- Each test class interacts with corresponding page objects to perform actions and validations.
- Assertions are used to validate expected outcomes.
- Logging is integrated to track execution flow.

- ```
  <!DOCTYPE suite SYSTEM "https://testng.org/testng-1.0.dtd">
  <suite name="IPL Suite for Chrome">
    <listeners>
      <listener class-name="com.ipl.automation.utils.TestListener"/>
    </listeners>
    <parameter name="browser" value="chrome"/>
    <test name="IPL Tests for Chrome">
      <classes>
        <class name ="tests.FooterLinksTest"/>
        <class name ="tests.TeamsTest"/>
        <class name ="tests.PointsTableTest"/>
        <class name ="tests.SearchTest"/>
        <class name ="tests.VenuesTest"/>
        <class name ="tests.AuctionTest"/>
        <class name ="tests.StatsTest"/>
      </classes>
    </test>
  </suite>
  ```

5. UTILITY CLASSES IMPLEMENTATION

Reusable utility classes are created to simplify common operations:

- ElementUtils → Handles element interactions (click, type, hover, JS actions)
- WaitUtils → Implements explicit waits for synchronization
- ScreenshotUtils → Captures screenshots on failure
- ConfigReader → Reads configuration properties
- JsonDataProvider → Provides data for data-driven testing

These utilities improve reusability and reduce code duplication.

Elements interaction through utils:

```
public class ElementUtils {
   WebDriver driver;
   WaitUtils wait;
   public ElementUtils(WebDriver driver) {
      this.driver = driver;
      this.wait = new WaitUtils(driver);
   }

   public void click(By locator) {
```
```

```
        wait.waitForElementClickable(locator).click();
    }

    public void type(By locator, String text) {
        wait.waitForElementVisible(locator).sendKeys(text);
    }

    public String getText(By locator) {
        return wait.waitForElementVisible(locator).getText();
    }

    public void hover(By locator) {
        Actions = new Actions(driver);
        actions.moveToElement(wait.waitForElementVisible(locator)).perform();
    }

    public void hover(WebElement element) {
        Actions = new Actions(driver);
        actions.moveToElement(element).pause(1500).perform();
    }

    public void clickUsingJS(By locator) {
        WebElement element = wait.waitForElementVisible(locator);
        JavascriptExecutor js = (JavascriptExecutor) driver;
        js.executeScript("arguments[0].click();", element);
    }

    public void scrollIntoView(By locator) {
        WebElement element = wait.waitForElementVisible(locator);
        JavascriptExecutor js = (JavascriptExecutor) driver;
        js.executeScript("arguments[0].scrollIntoView({block: 'center'});", element);
    }
```

6. USING WAITS FOR SYNCHRONIZATION:

```
public class WaitUtils {

WebDriver driver;
WebDriverWait wait;

public WaitUtils(WebDriver driver) {
    this.driver = driver;
    this.wait = new WebDriverWait(driver, Duration.ofSeconds(10));
}
```

```
public WebElement waitForElementVisible(By locator) {
   return wait.until(ExpectedConditions.visibilityOfElementLocated(locator));
}

public WebElement waitForElementClickable(By locator) {
   return wait.until(ExpectedConditions.elementToBeClickable(locator));
}

public void waitForElementClickable(WebElement element) {
   wait.until(ExpectedConditions.elementToBeClickable(element));
}

public WebElement waitForElementPresence(By locator) {
   return wait.until(ExpectedConditions.presenceOfElementLocated(locator));
}

public void waitForNewWindow(int previousWindowCount) {
   wait.until(driver -> driver.getWindowHandles().size() > previousWindowCount);
}

public List<WebElement> waitForElementsVisible(By locator) {
   try {
      return wait.until(
            ExpectedConditions.visibilityOfAllElementsLocatedBy(locator));
   } catch (Exception e) {
      return new ArrayList<>(); // safe fallback
   }
}
```

7. CAPTURING SCREENSHOTS ON FAILURE:

```
public class ScreenshotUtils {
   public static void capture(WebDriver driver, String testName) {

      TakesScreenshot ts = (TakesScreenshot) driver;
      File src = ts.getScreenshotAs(OutputType.FILE);

      try {
         FileUtils.copyFile(src, new File("screenshots/" + testName + ".png"));
      } catch (IOException e) {
         e.printStackTrace();
      }
   }
```

8. DATA-DRIVEN TESTING:

- Test data is stored in JSON format (testdata.json).
- The JsonDataProvider class reads and provides data to test cases.
- This enables dynamic validation of UI data against expected values.

```
public class JsonDataProvider {
    static JsonObject data;
    static {
        try {
            FileReader reader = new FileReader("src/test/resources/testdata.json");
            data = JsonParser.parseReader(reader).getAsJsonObject();
        } catch (Exception e) {
            e.printStackTrace();
        }
    }

    public static String[] getTeamNames() {

        JsonArray teams = data.getAsJsonArray("teams");
        String[] teamNames = new String[teams.size()];

        for (int i = 0; i < teams.size(); i++) {
            teamNames[i] = teams.get(i).getAsString();
        }

        return teamNames;
    }

    public static String getTopTeamName() {
        return data.getAsJsonObject("topTeam").get("name").getAsString();
    }

    public static int getTopTeamMatches() {
        return data.getAsJsonObject("topTeam").get("matches").getAsInt();
    }

    public static int getTopTeamPoints() {
        return data.getAsJsonObject("topTeam").get("points").getAsInt();
    }

}
```

9. LOGGING & REPORTING:

LOGGING IMPLEMENTATION:

- Log4j is configured using log4j2.xml.
- Logs are generated during test execution and stored in automation.log.
- Helps in debugging and analyzing test flow.
- Logging XML file:

```
<?xml version="1.0" encoding="UTF-8"?>
<Configuration status="WARN">
  <Appenders>
    <!-- Console Output -->
    <Console name="Console" target="SYSTEM_OUT">
      <PatternLayout pattern="%d{HH:mm:ss} [%t] %-5level %logger - %msg%n"/>
    </Console>
    <!-- File Output -->
    <File name="FileLogger" fileName="logs/automation.log">
      <PatternLayout pattern="%d{yyyy-MM-dd HH:mm:ss} %-5level %logger -
%msg%n"/>
    </File>
  </Appenders>
  <Loggers>
    <Root level="info">
      <AppenderRef ref="Console"/>
      <AppenderRef ref="FileLogger"/>
    </Root>
  </Loggers>
</Configuration>
```

- Log Reports:

```
2026-04-10 15:14:53 INFO  tests.FooterLinksTest - ---------------Case 1: Footer Links
Test---------------
2026-04-10 15:14:53 INFO  tests.FooterLinksTest - Opening Home Page
2026-04-10 15:14:53 INFO  tests.FooterLinksTest - Scrolling to footer
2026-04-10 15:14:53 INFO  tests.FooterLinksTest - Validating TEAM section links
2026-04-10 15:14:53 INFO  tests.FooterLinksTest - Validating ABOUT section links
2026-04-10 15:14:53 INFO  tests.FooterLinksTest - Validating GUIDELINES section
links
2026-04-10 15:14:53 INFO  tests.FooterLinksTest - Validating CONTACT section links
2026-04-10 15:14:54 INFO  tests.FooterLinksTest - Footer validation completed
```

```
successfully
2026-04-10 15:14:54 INFO  tests.FooterLinksTest - ---------------Test Case
Completed---------------
2026-04-10 15:14:54 INFO  tests.FooterLinksTest -
2026-04-10 15:15:04 INFO  tests.TeamsTest - ---------------Case 2: Teams Hover
Test---------------
2026-04-10 15:15:04 INFO  tests.TeamsTest - Navigating to Teams page
2026-04-10 15:15:09 INFO  tests.TeamsTest - Total number teams listed: 10
2026-04-10 15:15:09 INFO  tests.TeamsTest - Validating team: Chennai Super Kings
2026-04-10 15:15:11 INFO  tests.TeamsTest - Team: Chennai Super Kings
2026-04-10 15:15:11 INFO  tests.TeamsTest - Winning years: CSK
2010 | 2011 | 2018 | 2021 | 2023
2026-04-10 15:15:11 INFO  tests.TeamsTest -
2026-04-10 15:15:11 INFO  tests.TeamsTest - Validating team: Delhi Capitals
2026-04-10 15:15:13 INFO  tests.TeamsTest - Team: Delhi Capitals
2026-04-10 15:15:13 INFO  tests.TeamsTest - Winning years: No trophies
2026-04-10 15:15:13 INFO  tests.TeamsTest -
2026-04-10 15:15:13 INFO  tests.TeamsTest - Validating team: Gujarat Titans
2026-04-10 15:15:15 INFO  tests.TeamsTest - Team: Gujarat Titans
2026-04-10 15:15:15 INFO  tests.TeamsTest - Winning years: GT
2022
2026-04-10 15:15:15 INFO  tests.TeamsTest -
2026-04-10 15:15:15 INFO  tests.TeamsTest - Validating team: Kolkata Knight Riders
2026-04-10 15:15:17 INFO  tests.TeamsTest - Team: Kolkata Knight Riders
2026-04-10 15:15:17 INFO  tests.TeamsTest - Winning years: KKR
2012 | 2014 | 2024
2026-04-10 15:15:17 INFO  tests.TeamsTest -
2026-04-10 15:15:17 INFO  tests.TeamsTest - Validating team: Lucknow Super Giants
2026-04-10 15:15:19 INFO  tests.TeamsTest - Team: Lucknow Super Giants
2026-04-10 15:15:19 INFO  tests.TeamsTest - Winning years: No trophies
2026-04-10 15:15:19 INFO  tests.TeamsTest -
2026-04-10 15:15:19 INFO  tests.TeamsTest - Validating team: Mumbai Indians
2026-04-10 15:15:21 INFO  tests.TeamsTest - Team: Mumbai Indians
2026-04-10 15:15:21 INFO  tests.TeamsTest - Winning years: MI
2013 | 2015 | 2017 | 2019 | 2020
2026-04-10 15:15:21 INFO  tests.TeamsTest -
2026-04-10 15:15:21 INFO  tests.TeamsTest - Validating team: Punjab Kings
2026-04-10 15:15:23 INFO  tests.TeamsTest - Team: Punjab Kings
2026-04-10 15:15:23 INFO  tests.TeamsTest - Winning years: No trophies
2026-04-10 15:15:23 INFO  tests.TeamsTest -
2026-04-10 15:15:23 INFO  tests.TeamsTest - Validating team: Rajasthan Royals
```

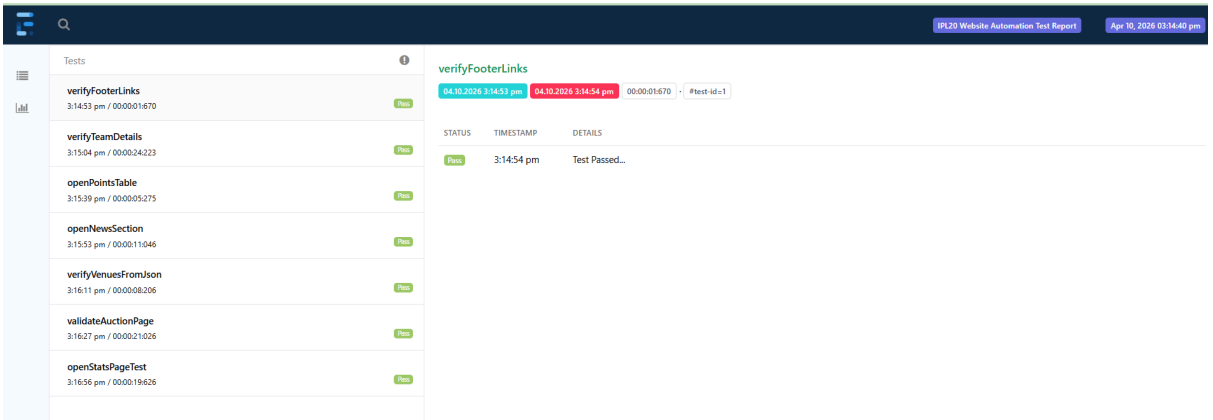
```
2026-04-10 15:15:25 INFO  tests.TeamsTest - Team: Rajasthan Royals
2026-04-10 15:15:25 INFO  tests.TeamsTest - Winning years: RR
2008
2026-04-10 15:15:25 INFO  tests.TeamsTest -
2026-04-10 15:15:25 INFO  tests.TeamsTest - Validating team: Royal Challengers
Bengaluru
2026-04-10 15:15:27 INFO  tests.TeamsTest - Team: Royal Challengers Bengaluru
2026-04-10 15:15:27 INFO  tests.TeamsTest - Winning years: RCB
2025
2026-04-10 15:15:27 INFO  tests.TeamsTest -
2026-04-10 15:15:27 INFO  tests.TeamsTest - Validating team: Sunrisers Hyderabad
2026-04-10 15:15:29 INFO  tests.TeamsTest - Team: Sunrisers Hyderabad
2026-04-10 15:15:29 INFO  tests.TeamsTest - Winning years: SRH
2016
2026-04-10 15:15:29 INFO  tests.TeamsTest -
2026-04-10 15:15:29 INFO  tests.TeamsTest - ---------------Test Case
Completed---------------
2026-04-10 15:15:29 INFO  tests.TeamsTest -
2026-04-10 15:15:39 INFO  tests.PointsTableTest - ---------------Case 3: Points Table
Test---------------
2026-04-10 15:15:39 INFO  tests.PointsTableTest - Clicking on Points Table
2026-04-10 15:15:44 INFO  tests.PointsTableTest - Fetching top team details from
Points Table
2026-04-10 15:15:44 INFO  tests.PointsTableTest - Top Team: RR
2026-04-10 15:15:44 INFO  tests.PointsTableTest - Matches Played: 3
2026-04-10 15:15:44 INFO  tests.PointsTableTest - Points: 6
2026-04-10 15:15:44 INFO  tests.PointsTableTest - ---------------Test Case
Completed---------------
2026-04-10 15:15:44 INFO  tests.PointsTableTest -
2026-04-10 15:15:53 INFO  tests.SearchTest - ---------------Case 4: Search
Test---------------
2026-04-10 15:15:53 INFO  tests.SearchTest - Navigating to News section
2026-04-10 15:15:57 INFO  tests.SearchTest - Performing search for: Auction 2026
2026-04-10 15:16:03 INFO  tests.SearchTest - Validating search results for relevance
2026-04-10 15:16:03 INFO  tests.SearchTest - Clicking on first search result
2026-04-10 15:16:04 INFO  tests.SearchTest - ---------------Test Case
Completed---------------
2026-04-10 15:16:04 INFO  tests.SearchTest -
2026-04-10 15:16:11 INFO  tests.VenuesTest - ---------------Case 5: Teams Hover
Test---------------
2026-04-10 15:16:19 INFO  tests.VenuesTest - Navigated to Venues page
```

```
2026-04-10 15:16:19 INFO  tests.VenuesTest - Venues data fetched
2026-04-10 15:16:19 INFO  tests.VenuesTest - Json data for venues fetched
2026-04-10 15:16:19 INFO  tests.VenuesTest - Venue details are being verified
2026-04-10 15:16:19 INFO  tests.VenuesTest - All venues validated successfully
2026-04-10 15:16:19 INFO  tests.VenuesTest - ---------------Test Case
Completed---------------
2026-04-10 15:16:19 INFO  tests.VenuesTest -
2026-04-10 15:16:27 INFO  tests.AuctionTest - ---------------Case 6: Auction Page
Test---------------
2026-04-10 15:16:27 INFO  tests.AuctionTest - Opening Auction Page
2026-04-10 15:16:42 INFO  tests.AuctionTest - Accepting cookies if present
2026-04-10 15:16:42 INFO  tests.AuctionTest - Waiting for page to load
2026-04-10 15:16:44 INFO  tests.AuctionTest - Scrolling down to load auction data
2026-04-10 15:16:46 INFO  tests.AuctionTest - Clicking on Top Buys section
2026-04-10 15:16:48 INFO  tests.AuctionTest - Fetching top buy player details
2026-04-10 15:16:48 INFO  tests.AuctionTest - Top Buy Player: Cameron Green
2026-04-10 15:16:48 INFO  tests.AuctionTest - Price: 252000000
2026-04-10 15:16:48 INFO  tests.AuctionTest - ---------------Test Case
Completed---------------
2026-04-10 15:16:48 INFO  tests.AuctionTest -
2026-04-10 15:16:56 INFO  tests.StatsTest - ---------------Case 7: Stats Page
Test---------------
2026-04-10 15:16:56 INFO  tests.StatsTest - Opening Stats Page
2026-04-10 15:17:11 INFO  tests.StatsTest - Accepting cookies if present
2026-04-10 15:17:11 INFO  tests.StatsTest - Scrolling down to load stats
2026-04-10 15:17:14 INFO  tests.StatsTest - Waiting for data to load
2026-04-10 15:17:16 INFO  tests.StatsTest - Fetching top player details
2026-04-10 15:17:16 INFO  tests.StatsTest - Top Player: Yashasvi Jaiswal
RR
2026-04-10 15:17:16 INFO  tests.StatsTest - Runs: 170
2026-04-10 15:17:16 INFO  tests.StatsTest - ---------------Test Case
Completed---------------
2026-04-10 15:17:16 INFO  tests.StatsTest -
```

REPORTING IMPLEMENTATION:

- Extent Reports is integrated using ExtentManager and TestListener.
- Automatically generates a detailed HTML report after execution.

- Displays test status (pass/fail), execution time, and logs.



10. CROSS-BROWSER EXECUTION:

- TestNG XML files (testng-chrome.xml, testng-edge.xml) are used to run tests on different browsers.
- Browser type is passed as a parameter and handled by DriverFactory.

  Results:

  The developed UI Automation Framework was successfully implemented and executed on the IPL website to validate multiple critical functionalities. The framework demonstrated stable performance, accurate validation, and efficient execution across all test scenarios.

TEST EXECUTION RESULTS:

- Total Test Cases Executed: **7**
- Test Cases Passed: **7**
- Test Cases Failed: **0**
- Execution Status: **100% Pass Rate**

All automated test cases executed successfully without any failures, indicating that the framework is reliable and the tested functionalities are working as expected.

KEY OUTCOMES:

1. **SUCCESSFUL AUTOMATION OF CRITICAL SCENARIOS**
   Core functionalities such as footer links, team details, points table, news search, venues validation, auction data, and stats page were automated and validated effectively.

2. **ROBUST AND STABLE FRAMEWORK**
   The use of dynamic waits and reusable utilities ensured stable execution without flaky test failures.

3. **EFFICIENT TEST EXECUTION**
   Automation significantly reduced manual effort and enabled faster validation of application features.

4. **ACCURATE DATA VALIDATION**
   JSON-based data-driven testing ensured that UI data matched expected values, improving test reliability.

5. **CROSS-BROWSER COMPATIBILITY VERIFIED**
   Tests were successfully executed on multiple browsers (Chrome and Edge), ensuring consistent behavior across environments.

6. **COMPREHENSIVE REPORTING AND LOGGING**
   Extent Reports provided detailed execution reports, while Log4j logs helped in tracking execution flow and debugging.

7. **EFFECTIVE FAILURE HANDLING MECHANISM**
   Automatic screenshot capture for failed test cases improved defect analysis and debugging efficiency.

8. **SCALABLE AND MAINTAINABLE FRAMEWORK**
   The modular architecture allows easy addition of new test cases and features without impacting existing code.

TEAM CONTRIBUTION:

This project was collaboratively developed by a team of two members, with responsibilities shared to ensure efficient development and timely completion of the automation framework.

CONTRIBUTION DETAILS:

J Doni Naga Siva:

- Developed core utility classes like ElementUtils, TestListner, ScreenShotUtils.
- Implemented Page classes and Test cases for Teams, Auctions and Stats tests and executed them.
- Worked on data-driven testing using JSON.
- Integrated Extent Reports for reporting of test cases.
- Implemented ScreenShot functionality for debugging purposes.

K Vamsi Krishna Varma:

- Developed base classes such as driver management and test setup.
- Developed core utility classes like WaitUtils, JsonDataProvider, ConfigReader.
- Implemented Page classes and Test cases for FooterLinks, Points Table, Search, Venues tests and executed them.
- Worked on data-driven testing using JSON.
- Integrated logging functionality using Log4j for better debugging.

COLLABORATIVE WORK:

- Requirement analysis and test case design.
- Designing overall framework architecture and project structure.
- Developed Page Object Model (POM) classes for different web pages.
- Implemented automated test execution cases using TestNG.
- Debugging and fixing issues.
- Enhancing framework features and stability.
- Preparing documentation and final reports.

CONCLUSION:

The UI Automation Testing Framework developed for the IPL website successfully demonstrates an industry-standard approach to automating web application testing. By leveraging technologies such as Selenium WebDriver, TestNG, and Maven, along with the Page Object Model (POM) design pattern, the framework ensures a clear separation of concerns, improved code reusability, and ease of maintenance.

The framework effectively automates critical user scenarios, validates dynamic data using JSON-based data-driven testing, and supports cross-browser execution. The integration of logging (Log4j), reporting (Extent Reports), and screenshot capture mechanisms enhances debugging capabilities and provides clear insights into test execution results.

The successful execution of all test cases with a 100% pass rate indicates the stability and reliability of both the framework and the tested functionalities. Additionally, the modular and scalable design allows for easy extension of the framework to accommodate new test cases and features in the future.

Overall, this project highlights the importance of automation in improving testing efficiency, reducing manual effort, and ensuring high-quality software delivery. It serves as a solid foundation for implementing advanced automation practices in real-world applications.

FUTURE ENHANCEMENTS:

The current automation framework provides a strong foundation; however, it can be further enhanced with advanced features to improve scalability, efficiency, and real-world applicability.

1. PARALLEL TEST EXECUTION

- Implement parallel execution using TestNG to run multiple test cases simultaneously.
- Reduces execution time and improves efficiency.

2. CI/CD INTEGRATION

- Integrate the framework with tools like Jenkins or GitHub Actions.
- Enables automated test execution on every code commit.

3. CLOUD-BASED TESTING

- Extend support to cloud platforms like Selenium Grid, BrowserStack, or Sauce Labs.
- Allows testing across multiple devices, browsers, and operating systems.

4. ENHANCED REPORTING

- Improve reporting by adding screenshots, logs, and step-level details directly into Extent Reports.
- Provide more insights into test execution.

5. API TESTING INTEGRATION

- Combine UI automation with API testing using tools like RestAssured.
- Enables end-to-end testing of the application.

6. DATABASE VALIDATION

- Integrate database testing to validate backend data against UI results.
- Ensures data consistency and integrity.

7. BDD IMPLEMENTATION

- Implement Behavior-Driven Development (BDD) using Cucumber.
- Makes test cases more readable and understandable for non-technical stakeholders.

8. RETRY MECHANISM FOR FAILED TESTS

- Add retry logic to automatically re-run failed test cases.
- Helps handle flaky tests and improves reliability.